LevelUp Cloud Trainings
Sateesh Pabbathi | Microsoft Certified Trainer (MCT)

Big Data World

A Complete Beginner's Guide

From Zero to Hero — Understanding Hadoop, HDFS, MapReduce & the Ecosystem

Big Data Basics	HDFS Architecture	MapReduce	Ecosystem
What & Why	Storage Layer	Processing Engine	Tools & Stack

www.levelupedu.net

ABOUT THE AUTHOR

Microsoft Certified Trainer | Data Engineering Educator

Sateesh Pabbathi

MCT

www.levelupedu.net

Sateesh Pabbathi

Microsoft Certified Trainer (MCT)

With 22+ years of industry experience in data and cloud technologies, Sateesh Pabbathi is a renowned Microsoft Certified Trainer (MCT) and the founder of LevelUp Cloud Trainings — India's premier data engineering education platform.

Having trained 5,000+ students across India, Sateesh is known for his unique story-driven, analogy-based teaching methodology that makes complex Big Data concepts crystal clear — even for absolute beginners.

His trademark teaching style uses everyday Indian business scenarios — Zomato, Flipkart, HDFC Bank, TCS, Infosys — to explain what could otherwise be dry, intimidating technology concepts.

Expertise & Certifications

- Microsoft Fabric | Azure Data Factory | Databricks
- Apache Spark & PySpark | GCP Data Engineering | AWS
- SQL Server | Python | Big Data & Hadoop Ecosystem
- DP-600 Certified | Microsoft Certified Trainer (MCT)

"I don't just teach technology — I build careers."

5,000+	22+	10+
Students Trained	Years Experience	Technologies Taught

Module 1: Welcome to the Big Data World

Before we dive into technology, let us understand WHY Big Data exists in the first place. Think like a business owner, not a developer!

> **Sateesh Says:**
> *Imagine you run Zomato. On New Year's Eve, 5 crore people open the app at the same time. Each one searches for food, clicks on restaurants, reads reviews, places orders, tracks delivery. How many records is that? Hundreds of crores of rows — per hour! Traditional databases will crash.*
> *This is exactly why Big Data was born. Welcome to the world where DATA is the new OIL!*

1.1 What is Big Data?

Big Data is simply data that is too large, too fast, or too complex for traditional tools (like MySQL or Excel) to handle efficiently. It is not just about SIZE — it is about the CHALLENGE of processing it.

1.2 The 5 V's of Big Data — Your Master Framework

Every Big Data interview and every real project revolves around these 5 V's. Memorise these!

The 5 V's of Big Data

```
+-------------------------------------------------------------------+
|                    THE 5 V's of BIG DATA                          |
+----------+--------------+-------------------------------------+
|  V       |  Meaning     |  Real-World Example (India)         |
+----------+--------------+-------------------------------------+
|  VOLUME  |  How MUCH    |  Flipkart: 10M orders on Big Billion |
|          |  data        |  Day stored every second            |
+----------+--------------+-------------------------------------+
|  VELOCITY |  How FAST    |  HDFC Bank: 50,000 credit card      |
|          |  data moves  |  transactions per second (real-time) |
+----------+--------------+-------------------------------------+
|  VARIETY |  How MANY    |  Swiggy: Text reviews + Food images |
|          |  data types  |  + GPS coordinates + Voice orders   |
+----------+--------------+-------------------------------------+
|  VERACITY |  How         |  TCS HR Data: Is this employee      |
|          |  ACCURATE    |  record correct? Is DOB valid?      |
+----------+--------------+-------------------------------------+
|  VALUE   |  What        |  Ola: Predict surge pricing to      |
|          |  INSIGHT     |  maximize driver earnings by zone   |
+----------+--------------+-------------------------------------+
```

1.3 Traditional DB vs Big Data — The Stadium Analogy

> **Sateesh Says:**
> *Think of a Traditional Database as a small Dhaba. It serves 50 customers well. Perfect food, fast service.*
> *Now think of Big Data as a stadium canteen during an IPL final — 1 lakh people need food simultaneously!*
> *The Dhaba model collapses. You need a distributed, parallel cooking system. That is Hadoop!*

Traditional Database (MySQL/SQL Server)	Big Data System (Hadoop/Spark)
Stores GBs of data	Stores Petabytes (1 PB = 1000 TB)
Single powerful server	Thousands of commodity machines
Structured data (tables only)	Structured + Semi + Unstructured
Scale UP (buy expensive hardware)	Scale OUT (add cheap machines)
ACID transactions first	Process first, validate later
Fast for small queries	Fast for massive batch processing
MySQL, Oracle, SQL Server	Hadoop, Spark, Hive, HBase

Module 2: HDFS — The Distributed Storage Giant

Sateesh Says:

Imagine Infosys stores millions of employee files in ONE big office cabinet. If that cabinet burns down — data is GONE.

HDFS says: Split every file into blocks. Store each block on MULTIPLE machines. Even if 2 machines die — data is SAFE!

This is exactly how Amazon stores your purchase history — no single point of failure!

2.1 What is HDFS?

HDFS stands for Hadoop Distributed File System. It is the storage backbone of the Hadoop ecosystem. HDFS is designed to store very large files across a cluster of commodity machines with automatic replication for fault tolerance.

2.2 HDFS Key Concepts

- **Block Size:** Files are split into blocks (default 128 MB). A 1 GB file = 8 blocks.
- **Replication Factor:** Each block is stored on 3 machines by default (configurable).
- **Fault Tolerance:** If one DataNode crashes, data is read from another replica automatically.
- **Write Once, Read Many:** HDFS is optimized for large sequential reads, not random updates.

2.3 HDFS Architecture — The Master-Slave Model

HDFS follows a Master-Slave architecture with one NameNode (master) and many DataNodes (slaves).

HDFS Architecture Diagram

```
   CLIENT (Your Laptop / Application)
       |
       |  Step 1: 'I want to read file /data/orders.csv'
       v
  +--------------------+
  |     NameNode       |  <-- THE MASTER (Brain of HDFS)
  |  (Metadata Only)   |      Knows WHERE every block lives
  |                    |      Does NOT store actual data!
  |  File: orders.csv  |
  |  Block 1 -> DN1,DN3|      Stores: File names, permissions,
  |  Block 2 -> DN2,DN4|      block locations, replication info
```

```
|  Block 3 -> DN1,DN2|
+--------------------+
     |
     |  Step 2: NameNode returns block locations to client
     |
     v
CLIENT reads directly from DataNodes (bypasses NameNode)
     |
+---------+   +---------+   +---------+   +---------+
|DataNode1|   |DataNode2|   |DataNode3|   |DataNode4|
| (DN1)   |   | (DN2)   |   | (DN3)   |   | (DN4)   |
|---------|   |---------|   |---------|   |---------|
|Block1   |   |Block2   |   |Block1   |   |Block2   |
|Block3   |   |Block3   |   |         |   |         |
+---------+   +---------+   +---------+   +---------+
Machine A     Machine B     Machine C     Machine D
(Rack 1)      (Rack 1)      (Rack 2)      (Rack 2)

Note: Replication Factor = 2 in above example (simplicity)
      In production: default Replication Factor = 3
```

2.4 How File Replication Works — The Flipkart Warehouse Analogy

> **Sateesh Says:**
>
> *Flipkart doesn't keep ONE copy of its bestselling iPhone in ONE warehouse. They stock it in Mumbai, Delhi AND Chennai.*
>
> *Why? Because if Delhi warehouse catches fire, Mumbai and Chennai still have stock!*
>
> *HDFS does the SAME for data. Default = 3 copies across different racks = 3 'warehouses'.*
>
> *Even if an entire rack (Rack 1) goes down — Rack 2 copies are still there. Your data is SAFE!*

HDFS Replication Strategy (Default Factor = 3)

```
Single File: 'sales_report.csv' (400 MB)
Splits into: Block A (128MB) + Block B (128MB) + Block C (144MB)

RACK 1 (Mumbai Data Center)    RACK 2 (Pune Data Center)
+---------------------------+  +---------------------------+
| DN1: [Block-A] [Block-B]  |  | DN3: [Block-A] [Block-C]  |
| DN2: [Block-B] [Block-C]  |  | DN4: [Block-C]            |
+---------------------------+  +---------------------------+

Block-A replicas: DN1, DN3 (cross-rack = fault tolerant!)
Block-B replicas: DN1, DN2
Block-C replicas: DN2, DN3, DN4

If ENTIRE RACK 1 goes down -> Rack 2 still has all blocks!
```

2.5 NameNode — The City's Address Book

The NameNode stores ONLY metadata — it never stores actual data. Think of it as a city's address book that knows exactly which house (DataNode) has which goods (blocks). Without the address book, you cannot find anything!

Key NameNode Facts (High Exam Weight!)

* NameNode is a SPOF (Single Point of Failure) in older Hadoop versions
* Hadoop 2.x+ introduced Secondary NameNode & NameNode HA (High Availability)
* Secondary NameNode is NOT a backup — it merges edit logs (checkpoint node)
* NameNode HA uses Zookeeper for automatic failover to Standby NameNode
* Stores: FsImage (snapshot) + EditLog (changes) in local disk
* Heartbeat: DataNodes send heartbeat every 3 seconds to NameNode

Module 3: MapReduce — Divide, Conquer & Combine

Sateesh Says:

Imagine you are the manager of an Infosys project. 1 crore code lines need to be reviewed for bugs.
You don't do it alone! You split the work: 1000 developers review 1000 lines each — PARALLEL!
Then you collect their reports and COMBINE: 'Total bugs = sum of all reviews'.
That is EXACTLY MapReduce! MAP = split and process. REDUCE = combine and summarise.

3.1 The MapReduce Programming Model

MapReduce is a programming model for processing large datasets in parallel across a distributed cluster. Google invented it in 2004. Hadoop's MapReduce is the open-source implementation. You write two functions: MAP and REDUCE.

3.2 MapReduce Pipeline — Step by Step

MapReduce Complete Pipeline

```
PROBLEM: Count how many orders each city got from Zomato data
Input File: orders.csv (10 GB, millions of rows)

INPUT DATA (Sample rows):
OrderID, City, Amount
1001, Mumbai, 350
1002, Delhi, 200
1003, Mumbai, 450
1004, Hyderabad, 300

STEP 1: INPUT SPLITTING
+-----------------------------------------------------------+
| HDFS splits file into blocks -> each block = one Map task |
|  Block1: rows 1-100k  |  Block2: rows 100k-200k | ...    |
+-----------------------------------------------------------+
                  |
                  v
STEP 2: MAP PHASE (parallel, one task per block)
+-----------------------------------------------------------+
| Mapper reads each row, emits KEY-VALUE pairs:             |
|                                                           |
| Input:  1001, Mumbai, 350                                 |
| Output: (Mumbai, 1)                                       |
|                                                           |
| Input:  1002, Delhi, 200                                  |
| Output: (Delhi, 1)                                        |
|                                                           |
```

```
| Input:  1003, Mumbai, 450                                |
| Output: (Mumbai, 1)                                      |
+----------------------------------------------------------+
                  |
                  v
STEP 3: SHUFFLE & SORT (framework handles this automatically)
+----------------------------------------------------------+
| Groups same keys together + sorts alphabetically        |
|                                                          |
| (Delhi,    [1, 1, 1, 1])                                |
| (Hyderabad,[1, 1])                                       |
| (Mumbai,   [1, 1, 1, 1, 1, 1, 1])                       |
+----------------------------------------------------------+
                  |
                  v
STEP 4: REDUCE PHASE (one reducer per unique key)
+----------------------------------------------------------+
| Reducer sums up all values for each key:                 |
|                                                          |
| (Delhi,    4)   -> Delhi received 4 orders              |
| (Hyderabad,2)   -> Hyderabad received 2 orders          |
| (Mumbai,   7)   -> Mumbai received 7 orders             |
+----------------------------------------------------------+
                  |
                  v
STEP 5: OUTPUT -> Written back to HDFS
Delhi:      4
Hyderabad:  2
Mumbai:     7
```

3.3 MapReduce Word Count — The Hello World of Big Data

Every Big Data student starts here. Count occurrences of each word in a massive document. Here is the logic explained simply:

MapReduce Word Count Flow

```
Input text: 'I love India I love tech'

MAPPER output (each word -> count of 1):
(I, 1) (love, 1) (India, 1) (I, 1) (love, 1) (tech, 1)

After SHUFFLE & SORT (grouped by key):
(I, [1,1]) (India, [1]) (love, [1,1]) (tech, [1])

REDUCER output (sum all values per key):
(I, 2) (India, 1) (love, 2) (tech, 1)

Final Result in HDFS:    I=2, India=1, love=2, tech=1
```

3.4 YARN — The Resource Manager (Hadoop 2.x+)

Sateesh Says:

> *In Hadoop 1.x, MapReduce did TWO jobs: resource management AND data processing. Bad design!*
> *YARN (Yet Another Resource Negotiator) was introduced in Hadoop 2.x to split these responsibilities.*
> *YARN = Office Admin who manages meeting rooms (resources). MapReduce just focuses on doing work!*
> *This allowed Spark, Hive, and others to also run on the SAME cluster.*

YARN Architecture

```
+-------------------------------------------------------------+
|                   HADOOP CLUSTER                            |
|                                                             |
|  +---------------------------+                              |
|  |   ResourceManager (RM)    |   <-- The Cluster CEO        |
|  |  (Runs on Master Node)    |       Tracks ALL resources   |
|  |  Scheduler + AppMgr       |       Accepts job submissions |
|  +---------------------------+                              |
|          |         |         |                              |
|          v         v         v                              |
|  +--------+  +--------+  +--------+                        |
|  | Node   |  | Node   |  | Node   |   <-- Worker Nodes     |
|  |Manager1|  |Manager2|  |Manager3|      Each has CPU+RAM  |
|  |--------|  |--------|  |--------|                        |
|  |AppMstr |  |Contai- |  |Contai- |   AppMaster: per-job   |
|  |        |  |ner(Map)|  |ner(Red)|   Container: task unit |
|  +--------+  +--------+  +--------+                        |
|                                                             |
|  Applications that run on YARN: MapReduce, Spark, Hive, Pig |
+-------------------------------------------------------------+
```

Module 4: The Hadoop Ecosystem — Your Big Data Toolkit

Sateesh Says:

Hadoop alone is like a kitchen with only a gas stove — it can cook, but you need knives, grinder, mixer too!

The Hadoop Ecosystem is a collection of tools built ON TOP of HDFS + MapReduce + YARN.

Each tool solves a specific problem. A data engineer picks the RIGHT tool for the RIGHT job.

Let's meet your new best friends in the Big Data world!

4.1 The Ecosystem Overview — Layered Architecture

Hadoop Ecosystem — Full Stack View

```
+-------------------------------------------------------------------+
|                    QUERY & ACCESS LAYER                           |
|  +----------+ +----------+ +---------+ +----------+ +---------+  |
|  |   Hive   | |   Pig    | |  HBase  | | Impala   | |  Presto |  |
|  | (SQL on  | | (Script  | | (NoSQL  | |(Fast SQL)| |(Fast SQL|  |
|  |  Hadoop) | | on HDFS) | |  DB)    | |          | |)        |  |
|  +----------+ +----------+ +---------+ +----------+ +---------+  |
+-------------------------------------------------------------------+
|                   PROCESSING LAYER                                |
|  +---------------------------+ +------------------------------+  |
|  |      MapReduce            | |         Apache Spark         |  |
|  |  (Batch Processing)       | |  (Fast In-Memory Processing) |  |
|  +---------------------------+ +------------------------------+  |
+-------------------------------------------------------------------+
|               RESOURCE MANAGEMENT LAYER                           |
|  +-------------------------------------------------------+    |
|  |            YARN (Yet Another Resource Negotiator)         |    |
|  |           Manages CPU + Memory across cluster             |    |
|  +-------------------------------------------------------+    |
+-------------------------------------------------------------------+
|                     STORAGE LAYER                                 |
|  +-------------------------------------------------------+    |
|  |       HDFS (Hadoop Distributed File System)              |    |
|  |         Stores all data across cluster nodes              |    |
|  +-------------------------------------------------------+    |
+-------------------------------------------------------------------+
|                  INGESTION & MOVEMENT                             |
|  +----------+ +----------+ +---------+ +-------------------------+|
|  |  Sqoop   | |  Flume   | |  Kafka  | |     NFS / REST APIs    ||
|  |(DB->HDFS)| |(Log->HDFS| |(Streams)| |                        ||
|  +----------+ +----------+ +---------+ +-------------------------+|
+-------------------------------------------------------------------+
|              ORCHESTRATION & MANAGEMENT                           |
|  +---------+ +-----------+ +--------+ +--------+ +----------+   |
|  |  Oozie  | | Zookeeper | |Ambari  | |  Hue   | |  Ranger  |   |
|  |(Schedule| |(Coord.)   | |(Admin) | |(Web UI)| |(Security)|   |
|  +---------+ +-----------+ +--------+ +--------+ +----------+   |
+-------------------------------------------------------------------+
```

4.2 Tool-by-Tool Breakdown — Know Your Weapons

Tool	Category	What it Does	Indian Analogy
Hive	SQL Query	Write SQL on Hadoop. Translates SQL to MapReduce jobs automatically	Like a translator who converts your English to code for the machine
Pig	Scripting	High-level scripting language (Pig Latin) for data transformation	Like writing a recipe — step by step instructions for data cooking
HBase	NoSQL DB	Column-oriented NoSQL database on top of HDFS. Real-time random access	Like a railway reservation system — instant lookup by PNR number
Sqoop	Ingestion	Transfers data between RDBMS (MySQL/Oracle) and HDFS bidirectionally	Like a truck that moves goods from a shop (DB) to warehouse (HDFS)
Flume	Ingestion	Collects, aggregates and moves large amounts of log data to HDFS	Like a pipeline that continuously pumps water (logs) to a tank (HDFS)
Kafka	Streaming	Distributed message queue for real-time data streaming at massive scale	Like a railway track — data travels at high speed between producer & consumer
Spark	Processing	Fast in-memory processing engine — 100x faster than MapReduce	Like an electric car vs old diesel truck — same job, far faster!
Oozie	Workflow	Workflow scheduler to automate and sequence Hadoop jobs	Like a project manager who runs tasks in the right order, on schedule
Zookeeper	Coordination	Distributed coordination service — manages cluster synchronization	Like a traffic signal system — ensures no two jobs clash at intersection
Ranger	Security	Centralized security framework — manages authorization and audit	Like a security guard who checks who can access which data room

Module 5: End-to-End Big Data Pipeline — How it All Fits

Now let us see how all these components work TOGETHER in a real-world scenario. We will use a Flipkart-style e-commerce use case.

5.1 Real-World Pipeline — Flipkart Sales Analytics

End-to-End Big Data Pipeline

```
BUSINESS QUESTION: 'Which category sold the most on Big Billion Day?'

[1] DATA SOURCES
+------------------+  +------------------+  +----------------+
| MySQL Orders DB  |  | Server Log Files |  | Clickstream    |
| (Structured)     |  | (Semi-structured)|  | Events (JSON)  |
+--------+---------+  +--------+---------+  +-------+--------+
         |                     |                     |
         v                     v                     v
[2] INGESTION LAYER
+------------------+  +------------------+  +----------------+
|  Sqoop           |  |  Flume           |  |  Kafka         |
| (DB -> HDFS)     |  | (Logs -> HDFS)   |  | (Stream topic) |
+--------+---------+  +--------+---------+  +-------+--------+
         |                     |                     |
         +---------------------+---------------------+
                               |
                               v
[3] STORAGE LAYER
+-----------------------------------------------------------------+
|                    HDFS (Raw Zone)                              |
|  /data/orders/  /data/logs/  /data/events/                     |
+-----------------------------------------------------------------+
                               |
                               v
[4] PROCESSING LAYER
+---------------------------+  +-----------------------------+
|   Apache Hive             |  |   Apache Spark              |
| (SQL queries for reports) |  | (ML, real-time aggregations)|
+---------------------------+  +-----------------------------+
                               |
                               v
[5] SERVING LAYER
+-----------------------------------------------------------------+
|  HDFS (Processed Zone)  /  HBase  /  Data Warehouse             |
+-----------------------------------------------------------------+
                               |
                               v
[6] ANALYTICS & REPORTING
+------------------+  +------------------+  +----------------+
|  Power BI / Hue  |  |  Dashboards      |  |  ML Models     |
| (Business Users) |  | (Managers)       |  | (Data Sci.)    |
+------------------+  +------------------+  +----------------+
```

```
ANSWER: Electronics sold 47% of all Big Billion Day revenue!
```

5.2 Hadoop vs Spark — When to Use What

Sateesh Says:

Students always ask: If Spark is 100x faster, why use MapReduce at all?
Answer: Spark is like a sports car — fast but needs expensive RAM (memory).
MapReduce is like a bus — slow but carries massive load with minimal cost.
Use Spark for speed-critical work. Use MapReduce when you have petabytes and limited budget.

Hadoop MapReduce vs Apache Spark

```
Feature          MapReduce            Apache Spark
-----------------------------------------------------------------
Speed            Slow (disk I/O)       100x faster (in-memory)
Processing       Batch only            Batch + Stream + ML
Ease of Use      Complex Java code     Python/Scala/SQL friendly
Memory Usage     Low (disk based)      High (needs more RAM)
Fault Tolerance  High (disk backup)    High (lineage-based DAG)
Use Case         Huge batch ETL        Real-time, ML, fast queries
Storage          HDFS                  HDFS, S3, Azure ADLS
API              Java only (mainly)    Python, Scala, Java, SQL
When to use      1PB+ cheap batch      Speed-first workloads
```

Module 6: Interview Cheat Sheet & Quick Reference

Sateesh Says:

In Indian IT interviews — TCS, Infosys, Wipro, Capgemini — these questions come up 90% of the time.

Memorise these answers. I have framed them in simple language that any interviewer will love!

6.1 Top 10 Big Data Interview Questions

Q1 What is Big Data?

Data that is too large, fast, or complex for traditional tools. Characterised by 5 Vs: Volume, Velocity, Variety, Veracity, Value.

Q2 What is HDFS? How does it work?

Hadoop Distributed File System. Splits files into 128MB blocks, stores 3 copies (replication) across DataNodes. NameNode manages metadata; DataNodes store actual data.

Q3 What is the difference between NameNode and DataNode?

NameNode = Brain (stores metadata only — file names, block locations). DataNode = Worker (stores actual data blocks). NameNode coordinates; DataNode executes.

Q4 Explain MapReduce with an example.

MAP: Split data, process in parallel, emit key-value pairs. SHUFFLE: Group by key. REDUCE: Aggregate values per key. Example: Word count in millions of documents.

Q5 What is YARN?

Yet Another Resource Negotiator. Resource manager for Hadoop 2.x. Separates resource management from processing. Allows multiple engines (Spark, MapReduce) on same cluster.

Q6 What is the difference between Hive and HBase?

Hive: SQL-like querying tool on HDFS (batch, analytical). HBase: NoSQL real-time database on HDFS (random read/write, OLTP). Hive for reports; HBase for lookups.

Q7 Why is NameNode a single point of failure?

Hadoop 1.x had ONE NameNode. If it crashed, entire cluster was down. Solved in Hadoop 2.x with NameNode HA using Active/Standby pair with Zookeeper failover.

Q8 What is the default block size in HDFS?

128 MB in Hadoop 2.x (was 64MB in 1.x). Large blocks reduce NameNode metadata overhead and improve throughput for sequential reads.

Q9 What is the difference between Sqoop and Flume?

Sqoop: Structured data movement (RDBMS <-> HDFS). Flume: Unstructured/log data streaming (web servers -> HDFS). Sqoop = Pull. Flume = Push/Stream.

Q10 Why is Apache Spark faster than MapReduce?

Spark processes data IN MEMORY (RAM). MapReduce writes intermediate results to disk after each step. Memory is 100x faster than disk, so Spark wins on speed.

Module 7: MCT Quick Reference Card

Use this reference card when teaching. These are the most important numbers, terms and concepts for students.

Big Data Numbers Every Student Must Know

```
1 KB  = 1,024 Bytes
1 MB  = 1,024 KB    (a song ~ 5MB)
1 GB  = 1,024 MB    (a movie ~ 2GB)
1 TB  = 1,024 GB    (a hard drive ~ 1TB)
1 PB  = 1,024 TB    (Facebook stores 100 PB+)
1 EB  = 1,024 PB    (Google processes ~ 20 EB/day!)
1 ZB  = 1,024 EB    (World generates ~ 120 ZB/year by 2025)

HDFS Block Size     : 128 MB (default, was 64MB in Hadoop 1.x)
Replication Factor  : 3 (default) — stored on 3 DataNodes
Heartbeat Interval  : 3 seconds (DataNode -> NameNode)
Heartbeat Timeout   : 10 minutes before node marked dead
MapReduce Phases    : Input -> Split -> Map -> Shuffle -> Reduce -> Output
```

Key Terminology Glossary

Term	Definition
Cluster	Group of computers working together as a single system
Node	Individual machine (computer) in a cluster
Master Node	Controls and coordinates the cluster (NameNode, ResourceManager)
Slave/Worker Node	Does the actual data storage and processing (DataNode, NodeManager)
Commodity Hardware	Cheap, standard off-the-shelf servers (not enterprise-grade)
Horizontal Scaling	Adding more machines to increase capacity (Scale Out)
Fault Tolerance	System continues working even when some components fail
Replication	Storing multiple copies of data on different nodes
Block	Fixed-size chunk (128MB) that HDFS splits files into
Metadata	Data about data (file names, locations, permissions)
DAG	Directed Acyclic Graph — execution plan for Spark jobs
In-Memory Processing	Storing intermediate data in RAM (Spark) vs disk (MapReduce)

Sateesh Says:

> *Remember: Big Data is NOT about technology — it is about SOLVING BUSINESS PROBLEMS at scale!*
>
> *Zomato, Flipkart, Ola, HDFC Bank — they all face the same challenge: too much data, too little time.*
>
> *Your job as a Data Engineer is to build the pipelines that turn raw data into business gold.*
>
> *Start small: understand HDFS + MapReduce well. Everything else in the ecosystem builds on this foundation.*
>
> *All the best for your Big Data journey! — Sateesh Pabbathi, MCT | www.levelupedu.net*